

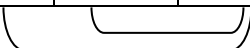
CS3364 Algorithms: Project 1

Find source reliability by counting inversions during sorting

In the context of the Web, the main applications include building meta-search engines, combining ranking functions, selecting documents based on multiple criteria. When we use Google search engine to search for information, Google always shows us a long list links one by one. Suppose Google considers combining ranking results from different sources. At the beginning they treat each source equally. In other words, they just sum all ranks from various sources for each web page and use the summation to generate the combined rank. However, they want to investigate the reliability of each source and assign high weight to the most reliable one in the future rank combination. Here we simply define reliability inversely proportional to the number of inversions between the rank from a source with the combined rank. That is, the source is more reliable if it has fewer inversions. Do you have the solution for Google? (Hint: this is a real-world application of sorting)

How do we count inversions? For an array A, when $i < j$ ($i, j \geq 0$) but $A[i] > A[j]$, this is an inversion. The sorting process is to reduce the number of inversions. For a complete sorted array/list, the inversion is 0. Here is an example (refer to the table below). In this example, there are two inversions in source A.

	Page8	Page5	Page6	Page1	Page3
Combined Rank	25	27	39	40	45
Source A	4	5	8	9	6



Requirement:

Sorting is one of the most broadly used fundamental operations in data processing. There are numerous sorting algorithms available. In this project, you are asked to modify the existing algorithms for real-world applications. You are required to modify three sorting algorithms for solving this problem: Two $O(n \log n)$ algorithms (merge sort and quick sort), and one algorithm (your choice).

Project Objective: In completing this project, you will further develop your ability to

1. Simplify and solve real world problems;
2. Apply the existing algorithms into real world applications;
3. Modify and implement algorithmic approaches;
4. Apply knowledge of asymptotic complexity analysis in algorithm selection.

Due Dates and Honor:

The submission deadline for this project will be announced on the TTU Canvas. This is a team project with a minimum of 3 and a maximum of 5 members. Please clearly specify the

contributions of each team member in the project report. It is very important that you understand and abide by the policy concerning programming projects.

Remember, your personal honor and integrity is far more important than your grade on a project.

Data for input: In this project, we assume there are only five sources to offer ranks for 10000 web pages. Each rank list is attached.

What to submit: You should submit the following documents to blackboard:

1. A report that documents your design and experiments, and
2. Source code of your program.

How it will be graded - Rubric

	Excellent (100%~90%)	Good (89%~80%)	Need Improvement (79%~60%)	Does Not Meet Requirements (59%~0%)
Designing correctness (40 pts)	All the three algorithms are designed completely correct	Two of three algorithms (mergesort and quicksort) are designed completely correct. One of them is reasonable.	Two of three algorithms (mergesort and quicksort) are designed completely correct.	Three algorithms (mergesort and quicksort) are designed reasonable correct.
Implementation Correctness (40 pts)	All the three algorithms are implemented completely correct, and produce correct solutions.	Two of the three algorithms (mergesort and quicksort) are implemented completely correct, and produce correct solutions. One of them is implemented most likely correct.	Two of the three algorithms (mergesort and quicksort) are implemented correctly, and produces correct solutions.	This program does not compile. Or some implementations are incorrect, not produce correct solutions, especially for mergesort and quicksort.
Readability (10 pts)	The code is exceptionally well organized and very easy to follow.	The code is fairly easy to read.	The code is readable only by someone who knows what it is supposed to be doing.	The code is poorly organized and very difficult to read.
Report Structure	The documentation	The documentation	The documentation	The documentation

(10 pts)	consists of four important components: interpretation of the given problem, methodology of the solution, experimental results, and conclusions. Each component is fully expanded. Besides, it also discussed future work and the team considerations	consists of four important components: interpretation of the given problem, methodology of the solution, experimental results, and conclusions. Each component is fully expanded.	consists of four important components: interpretation of the given problem, methodology of the solution, experimental results, and conclusions. But each component is partially expanded.	does not consist of four important components: interpretation of the given problem, methodology of the solution, experimental results, and conclusions.
----------	--	---	---	---

We will discuss the project in class. If you have any questions related to this project, please feel free to ask during class.